

[Download the article](#)

[Linux | Runoob tutorial \(runoob.com\)](#) is recommended as a starting point
[Linux Cookbook](#)

linux basics

1. Basic operations

1 ls list

`command [-options] parameter ...`

Options are usually given with a '-' sign, -h, and sometimes '--' is used to use the full name, --help

`ls -l` Display detailed information

`ls -a` Show hidden files

`ls -R` displays subdirectory files

`ls -F` displays file type

The -F option adds a forward slash (/) after the directory name to make it easier for users to distinguish in the output.

Similarly, it also adds an asterisk (*) after an executable file (such as the my_script file above) to help users find out which files can be run on the system

`ls -i` displays the file inode encoding

Wildcard

`ls -l t?st` replaces a character

`ls -l t*st` replaces multiple characters

`ls -l t[abc]st` Replace with abc

`ls -l t[a-c]st` Replace with a through c

`ls -l t[!d]st` is replaced with d

`pwd` print working directory

`pwd -P` If you are in a linked folder, simple pwd will appear in the wrong location. Add the -P option to get the correct path.

2 Process files

`touch test` creates an empty file

`rm -f` Forced deletion

`cp -R Document/ NewDocument/` The directory NewDocuments does not exist before the cp -R command is executed. It is created with the cp -R command and the entire contents of the Documents directory is copied into it.

`ln -s file symlink_file` **Create a soft link, equivalent to a shortcut**

`cp -i file1 file2` -i option is to ask when performing the operation

For example: `rm -i`

`mv -i`

`mkdir -p /path/to/new/dir` Create missing parent directories as needed. That is, if you only have /path, this command will automatically create the /path/to, /path/to/new, path/to/new/dir folders.

3 View file content

`file .bashrc` View file types and even the location of soft links

`cat -n file` View text and add line number

`less file` View large text

`tail -n 5 file` or `tail -5 file` View the last 5 lines

`head -n 5 file` or `head -5 file` View the first 5 lines

`tail -f file` This option allows you to view the contents of the file while it is being used by other processes. The tail command remains active and continuously displays the content added to the file. This is a great way to monitor system logs in real time.

2. Data Management

1 Data sorting

`sort file` By default, the sort command sorts the data lines in the text file according to the sorting rules of the default language specified by the session.

`sort -n file` Sort numbers by value in ascending order

`sort -nr file` Sort numbers in reverse order by value

`sort -M file` Files containing timestamp dates are sorted by the default sorting method

2Data search

The `grep [options] pattern [file]` grep command outputs the entire line matching the pattern

`grep -v t file` Output lines that do not contain t

`grep -n t file` outputs the line and line number containing t

`grep -c t file` Output the number of lines containing t

`grep -e t -e r file` Output lines containing t and r

3 command alias

`alias -p` View available aliases

`alias li='ls -i --color=auto'` creates aliases and uses color auto-encoding

`unalias li` Delete alias If the deleted alias is not set by you, the alias will appear again the next time you log in to the system. An alias can be permanently removed by modifying the environment file.

After this is modified in the .bashrc file, the command can be run every time the terminal is started, so that the alias does not have to be renamed every time. When entering a login command that is very long and does not need to be modified, specifying an alias will be an elegant operation

3. Environment variables

Set variables

Global environment variables are visible to the shell session and all spawned subshells. Local environment variables are only visible to the shell that created them.

- Set local variables

`my_variable="Hello World"` No spaces between variable name, equal sign, and value

- Set global variables

The `export my_variable="Hello World"` export command makes it a global environment variable.

This can be used to exchange variables between multiple scripts, which may be useful in large scripts.

-Remove environment variables

`unset my_variable`

4. Build basic scripts

To run two commands together, you can put them on the same line and separate them with semicolons.
For example:

```
1 #!/bin/bash
2 for i in {0..8}; do cd condor_primelv2_${i}; source SplitAndRun.sh; cd ..; done
```

1 echo command

- By default, there is no need to use quotes to delimit the string to be displayed

```
echo this is a test
```

- The echo command can use single quotes or double quotes to delimit strings `echo "this's a test"`

```
echo -n
```

 The string and command output are displayed on the same line, that is, no line breaks

2 Custom variables

- Variable names are case sensitive
- Use the equal sign to assign a value to a variable. **No spaces between variables, equal signs and values** `a=1;`

```
str="test"
```

3 Command Substitution

Extract information from command output and assign it to variables

- Backtick (```) *This can show you are a master, but it is more recommended to use the following one, because it is similar to other variable naming formats*
- `$( )` format

```
1 today=$(date +%y%m%d) #prints the date with two digits: 240116
2 cat log > log.$today
```

Tips

`yy` Copy a line

`p` Paste a line

`u` Undo the last operation

`.` Repeat last operation

`/test` Search test

`:` last line mode, more operations can be performed

`:1` locates the first line

`:$` locates the last line

`:. .+5w test.txt` The output of the five lines after the current line is test.txt

4 Mathematical operations

Use square brackets to assign the result of a mathematical operation to the variable `a=$(( (1+1) * 2 )` but the bash shell only supports integer operations

floating point calculation

- bc calculator
Access the bash calculator via `bc` and exit the calculator via `quit`
- use bc in scripts

```
1 a=$(echo "scale=4; 3.4 / 5" | bc) #scale sets four decimal places
```

- **Double brackets(())** Statements inside double brackets can use advanced mathematical expressions without escaping the greater than sign or the like >
- *I think in general, you can use double brackets without thinking, and just use bc for floating point numbers*

Exit status code

`echo $?` saves the exit status code of the last executed command, the status code without error is 0

5. Structured commands

1 if-then

```
1 if command #if the command's exit status is 0
2 then
3 commands #then the commands in the block are executed
4 else
5 commands #otherwise, execute the else branch
6 fi
7 #nested structure
8 if command1 then
9 commands elif command2 then
10 commands fi
```

- **Numerical comparison**

There are not many statements that use square brackets. Basically, except for conditional judgments, parentheses are used for everything else

```
1 if [ $n1 -gt $n2 ]
2 then echo $n1
3 else echo $n2
4 fi
```

condition	describe
<code>n1 -eq n2</code>	<code>n1 = n2</code>
<code>n1 -ge n2</code>	<code>n1 >= n2</code>
<code>n1 -gt n2</code>	<code>n1 > n2</code>
<code>n1 -le n2</code>	<code>n1 <= n2</code>
<code>n1 -lt n2</code>	<code>n1 < n2</code>
<code>n1 -ne n2</code>	<code>n1 != n2</code>

- **String comparison**

condition	describe
<code>str1 = str2</code>	Are str1 and str2 consistent
<code>str1 != str2</code>	Are str1 and str2 inconsistent
<code>str1 < str2</code>	**
<code>str1 > str2</code>	**
<code>-n str1</code>	Check whether the length of str1 is not 0
<code>-z str1</code>	Check whether the length of str1 is 0

- *Whether one string is greater or less than another string*
- The greater than and less than signs must be **escaped** `str1 \> str2`, otherwise the shell will treat them as redirection characters
- The size comparison is the Unicode encoding value of the character
- The order of greater than and less than is different from that used by the sort command
- **Boolean operator symbols**
 && and
 || or

Advanced Apps

- **Double brackets()** Statements inside double brackets can use advanced mathematical expressions without escaping the greater than sign or the like >
- **case command**

```

1 read choice
2 case $choice in
3     a)
4         command
5         ;;
6     b)
7         command
8         ;;
9     *)
10        echo "no choice"
11        ;;
12 esac

```

2 for

- for a in *list*

```

1 list=abc "ab'c" "ab c"
2 for test in abc "ab'c" "ab c"
3 do
4     command
5 done
6 file="list.txt"
7 for test in $(cat $file) #iterate using spaces, tabs, and newlines as field separators
8 IFS=$'\n' #use newlines as the separator, skipping spaces and tabs inside fields
9 IFS=$'\n:' #use newlines and colons as field separators

```

- for a in *range*

```

1 for ((a=1,b=10;a<=10;a++.b--)) #a C-like shell command

```

3 while and until

```

1 a=3
2 while echo $a #this only demonstrates that a command can be placed in while
3 [ $a -ge 0 ] #leave a space inside the square brackets
4 do
5     a=${a-1} #no spaces are allowed around assignment; alternatively use parentheses: a=$((a-1))
6 done
7 until echo $a
8 [ $a -eq 0 ]
9 do
10     a=${a-1}
11 done

```

4 break and continue

```

1 for ((a=1;a<4;a++))
2 do
3     if [ $a -eq 2 ]; then
4         continue #continue stops this iteration and starts the next one
5     fi
6     for ((b=1;b<4;b++))
7     do
8         if [ $b -eq 2 ]; then
9             break 2 #break normally stops one loop; 2 stops this loop and its outer loop
10        fi
11    done
12 done > test.txt #write the loop output to a file, or pipe it to another command such as: done | sort

```

5 read

`echo -n` The string and command output are displayed on the same line, that is, no line breaks

`read -p` can specify prompt characters

`read -t 5` If there is no input for more than 5 seconds, a non-0 exit status code will be returned.

`read -n 1` automatically exits and assigns a value when the number of characters entered is 1.

`read -s` will not display entered data, used for passwords

```

1 echo -n "enter your name:"
2 read name
3 read -p "enter your name:" first last #both forms work; this form can assign multiple variables
4 if read -t 5 -p "enter your name:" first last
5 then
6 else
7     echo "time out"
8 fi
9 read -n 1 -p "enter y/n" chose

```

6. Output and input

A command will display output on the terminal monitor. Correct output will point to the STDOUT file, and incorrect output will point to the STDERR file.

- standard file descriptor

0 standard input 1 standard output 2 standard error

`ls file1 file2 file3 1>out 2>err` 1 points to the correct output, 2 points to the incorrect output. If not specified, both correct and incorrect output will be output to the terminal.

`ls file1 file2 file3 &>both` & will output both

- redirect output

If you intentionally generate error messages in your script, you can redirect a single line of output to STDERR. This simply requires redirecting output to the STDERR file descriptor using the output redirection symbol. When redirecting to a file descriptor, you must precede the file descriptor index value with an &:

`echo "This is an error message" >&2`

- exec command

The exec command starts a new shell and redirects the STDOUT file descriptor to the specified file, such as `exec`

`2>error`

`exec 0<inputfile` This allows the script to get input in the file testfile instead of the keyboard. The previous read command read data 0 from STDIN, which is the data entered through the keyboard. Redirecting 0 to inputfile here means that the data will be read from here.

`exec 3>file3out` You can customize a descriptor so that the output you want to output can be placed within the 3 symbol later. `echo "This should be stored in a file" >&3` can maintain the normal output of the display and redirect specific information to a specified file (such as a log file)

- Fantastic little features

- `/dev/null` is like a trash can

`echo "This is an error message" > /dev/null` will discard the data

`cat /dev/null > testfile` This will directly clear the file contents without re-creating it.

- `mktemp testfile.XXXXXX` will fill in any characters and create a *file*

`mktemp -t testfile.XXXXXX` will create a file in the system's temporary directory `/tmp/test.XXXXXX`

`mktemp -d testdir.XXXXXX` will fill in any characters and create a *folder*

- `tee test` is like a T-to-interface

`data | tee test` will be output to the screen and the test file at the same time (>test)

`data | tee -a test` will not overwrite test(>>test)

7. Script running

1 System signals and generation

sig	value	mean
1	SIGHUP	hang
2	SIGINT	Interrupt -> Ctrl+C
3	SIGQUIT	Stop
9	SIGKILL	Unconditional termination -> kill -9 PID
20	SIGTSTP	Stop or pause without terminating -> Ctrl+Z

- trap command *trap command signals*

`trap "echo 'trapped Ctrl+C'" SIGINT` Entering Ctrl+C will not stop the script but execute the echo command, which prevents the command from being interrupted.

2 Script runs in the background

`&` Adding `&` after a command can make the command run in the background, and by redirecting 1 and 2, the terminal will not display information and achieve complete silence.

`ps` can view current job information

`jobs` It is more recommended to use the `jobs` command to check the job status and have some command line options.

`-l PID` | `-r running` | `-s stopped`

Tips: Plus and minus signs in `jobs` output. A job with a plus sign is the default job; if a job-control command does not specify a job number, it refers to that job. A job with a minus sign becomes the next default job after the current default job ends. At any time, there can be only one job with a plus sign and one with a minus sign.

8. Advanced scripts

- function

```

1  function func1 { #define a function
2      read -p "enter your number:" a
3      echo "$a *2" |bc
4  }
5  func1 #call func1 directly
6  result=$(func1) #store the function result in $result
7  echo -e "\nyour result is $result" #-e enables escape sequences such as \n
8
9  function func2 {
10     #local result=$(echo "$1 * $2" |bc) #local keeps the variable inside the function
11     echo "$1 * $2" |bc
12 }
13 if [ $# -eq 2 ]; then    #`$#` gives the number of arguments
14     result=$(func2 $1 $2) #for example, run: source test.sh 1.1 2
15     echo "your result is $result"
16 else
17     echo "error"
18 fi

```

- array

```

1  #input array
2  function func3 {
3      local myarray=( $(echo "$@") ) #`$@` expands the argument list

```

```

4     echo ${myarray[*]} #expand all array elements as one argument, separated by the first character
    of IFS
5 }
6 array=(1 2 3 4 5)
7 #array=("apple" "banana" "cat")
8 func3 ${array[*]}
9
10 #output array
11 function func4 {
12     local n=($#)
13     local myarray=( $(echo "$@" ) )
14     local newarray=( $(echo "$@" ) ) #create a newarray with the same size so its elements can be
    replaced in order
15     for (( i=1; i<=$n; i++ )) {
16         newarray[$i]=${myarray[$i]} * 2))
17     }
18     echo ${newarray[*]}
19 }
20 array=(1 2 3 4 5)
21 func4 ${array[*]}

```

- recursion

```

1 function func5 {
2     if [ $1 -eq 1 ]; then
3         echo $1
4     else
5         local x=$(( $1 - 1 )) #calculate n-1
6         local result=$(func5 $x)
7         echo $(( $result * $1 )) #
8     fi
9 }
10 read a
11 echo $(func5 $a)

```

9. sed command

It should be noted that the sed command will not modify the original file, but the output in the terminal will change. You can use redirection to output the sed-modified file or use the -i option to directly modify the file content

- Used to modify text in batches

`sed 's/old/new/' test.txt` s is the replacement command

`sed -e 's/old1/new1/; s/old2/new2/' test.txt` Multiple modifications, pay attention to semicolons and single quotes

`sed -f txt.sed test.txt` Put `s/old1/new1/` into `txt.sed` and use the -f option to make a lot of modifications

`sed 'd' test.txt` d is the delete command *Use with caution*

`sed 'i\text'` and `sed 'a\text'` add a line before and after the specified line respectively.

- Command group *If you need to execute multiple commands in a single line, you can use curly braces to group them together*

```

1 sed -n '/test/{
2 > =
3 > p
4 > }' test.txt

```

- Replacement flag

`sed 's/pattern/replacement/flags'` flag is a replacement flag. There are four types that can be written together.

- Number specifies the number in the text

`echo -e "test1 test1 test2 \ntest3 test4" | sed 's/test1/new/2'`

->test1 new test2

->test3 test4

-g replace all

`echo -e "test1 test1 test2 \ntest3 test4" | sed 's/test1/new/g'`

->new new test2

->test3 test4

-p prints out the replaced lines *-n means only print matching lines*

`echo -e "test1 test1 test2 \ntest3 test4" | sed -n 's/test1/new/gp'`

->new new test2

-w *file* writes the replacement results to a file

- `sed 's//ccc0|\nchain->Add("\(FILE_PATH\)")|g'` When / will cause confusion, you can use some other symbol to separate it
- Specify the row address *Same applies to i and a*
- `sed '2s/test1/new/'` specifies line 2
- `sed '2,4s/test1/new/'` specifies lines 2 to 4
- `sed '2,$s/test1/new/'` specifies 2 lines after \$ represents the last line
- Print
 - p *print text*
- = *Print line number*
- | *List lines, you can print out non-printable characters, such as \t*
- `sed -n '/test/p'` is similar to `grep test`. The former is more advanced and can use regular expressions. See Chapter 11.

10. gawk command

Is an enhanced version of awk

`gawk '{print "hello"}'` sets hello to display a fixed line of text string, so no matter what text is entered in the data stream, you will get the same text output

`echo "me:hello world" | awk '{print $1}' ->me:hello` uses spaces as delimiters by default

`echo "me:hello world" | awk -F: '{print $1}' ->me` **-F: You can specify: as the delimiter** -F[:/]' You can specify multiple characters

```

1  ---for test.gawk---
2  BEGIN {
3      print "title:character table"
4      print "character \t number"
5      print "---- \t ----"
6  }
7  {
8      print $1 "\t" $2
9  }
10 END {
11     print "end"
12 }
13 ---
14 gawk -f test.gawk test.txt #like sed's -f option, commands can be placed in a file

```

Some common processing

- Mathematical expressions

`gawk '$1 == 0 {print $0}'` means that if the first column is 0, the entire line will be output.

- if statement

`gawk '{if ($1 > 2) print $1}'`

`gawk '{if ($1 > 2) {a = $1 * 2 ; print a} else {print $2}}' test.txt`

- while,for statement *similar to if*

printf formatted printing

Need to add a newline character, otherwise it will be output on the same line

`printf "format string", var1, var2`

The format of *format string* is `%[modifier]control-letter`

`%d` or `%i` displays an integer

`%e`Scientific notation

`%f` displays floating point `%2.1f` retains one decimal place

`%s` displays the string

`gawk 'BEGIN{FS=","} {printf "%s ", $1} END{printf "\n"}'` is set as the delimiter and prints the characters in the first column. Since there is no line break, a line break is output at the end, so that the characters in the first column will be output on the same line.

`gawk 'BEGIN{FS="\n"; RS=""} {printf "%-16s %s\n", $1, $4}' data2` `%16s` means the output width is 16 characters right-aligned `%-16s` left-aligned

Ima Test

123 Main Street

Chicago, IL 60601

(312)555-1234

Frank Tester

456 Oak Street

Indianapolis, IN 46201

(317)555-9876

By default, RS is the newline character \n, indicating that the record is one line. Setting RS to blank here means that before a blank line appears, it is viewed as one line, with the newline character as the separator, and the 4th column is the phone column

11. Regular expressions

> Can be used in sed and gawk

- Anchor character

- `^` anchors the beginning of the line

`sed -n '/^this/p'` will only match lines starting with this

- `$` anchors end of line

`sed -n '/end$/p'` will only match lines ending with this

- Combine anchor points to delete blank lines

`sed '/^$/d'` **Highly recommended**

- period character

The dot character can match any single character except newline characters. The period character must match one

character

- character group []

```
echo "Yes" | sed -n '/[Yy]es/p'
```

```
sed -n '^[0-9][0-9]$/p'
```

 will match two digits

```
sed -n '^[^h]at$/p'
```

 ^ is placed in square brackets to exclude it.

- Asterisk *The asterisk indicates that the preceding character can appear 0 or more times*

```
echo "ik" | sed -n '/ie*k/p'
```

 can still be matched in this way

> Cannot be used in sed

- Question mark *Question mark indicates that the preceding character can appear 0 or 1 times*
- Plus sign *The plus sign indicates that the preceding character can appear 1 or more times*

- Curly brackets {m,n} *indicates that the preceding character can appear m to n times*

By default, gawk does not recognize regular expression intervals. You must specify the gawk command line option --re-interval.

```
echo "beet" | gawk --re-interval '/be{1}t/{print $0}'
```

- Vertical lines *Vertical lines represent or*

```
gawk '/cat|dog/{print $0}'
```

- Expression grouping ()

```
gawk '/(c|b)a(b|t)/{print $0}'
```

Postscript

Some messy notes, no guarantee of accuracy

du -sh .

sed -i "s/forward0/forward/g" crab3_forward0_Ntuple.py

```
grep -o "$FIELD" "$FILE" | wc -l
```

 Number of occurrences

```
grep -A 5 "$FIELD" "$FILE"
```

 appears 5 lines after

kill \$(jobs -p)

condor_rm -all

condor_rm -all -name lpcschedd6.fnal.gov Delete condor jobs on lpc

The ls -lhr command will list the detailed information of all files and subdirectories in the current working directory in human-readable form, in reverse order of modification time. This makes it easy to view recently modified files.

cat ion.C |wc determines the number of lines of ioc

s/old/new/gc replaces all occurrences of old in the entire file and prompts for each replacement.

When writing a shell script .sh file in the terminal, you can use the following method to achieve simultaneous indentation of multiple lines:

1. Use vim to open your shell script file, and then press the ESC key to enter command mode.
2. Press the ctrl+v keys to enter visual block mode, and then use the up and down keys to select the multiple lines of code you want to indent.
3. Press the shift+i keys to enter insert mode, and then continuously tap the space bar to add indentation (note that only one line will display indentation at this time).
4. Finally press the esc key and you will find that all selected lines are indented by 1.
If you want to remove spaces in front of multiple lines, you can follow these steps:
5. Also press the ctrl+v keys to enter the visual block mode, and then use the up, down, left and right keys to select the multiple lines of code you want to delete spaces and the preceding spaces.

6. Then press the del key to delete the selected space 1.

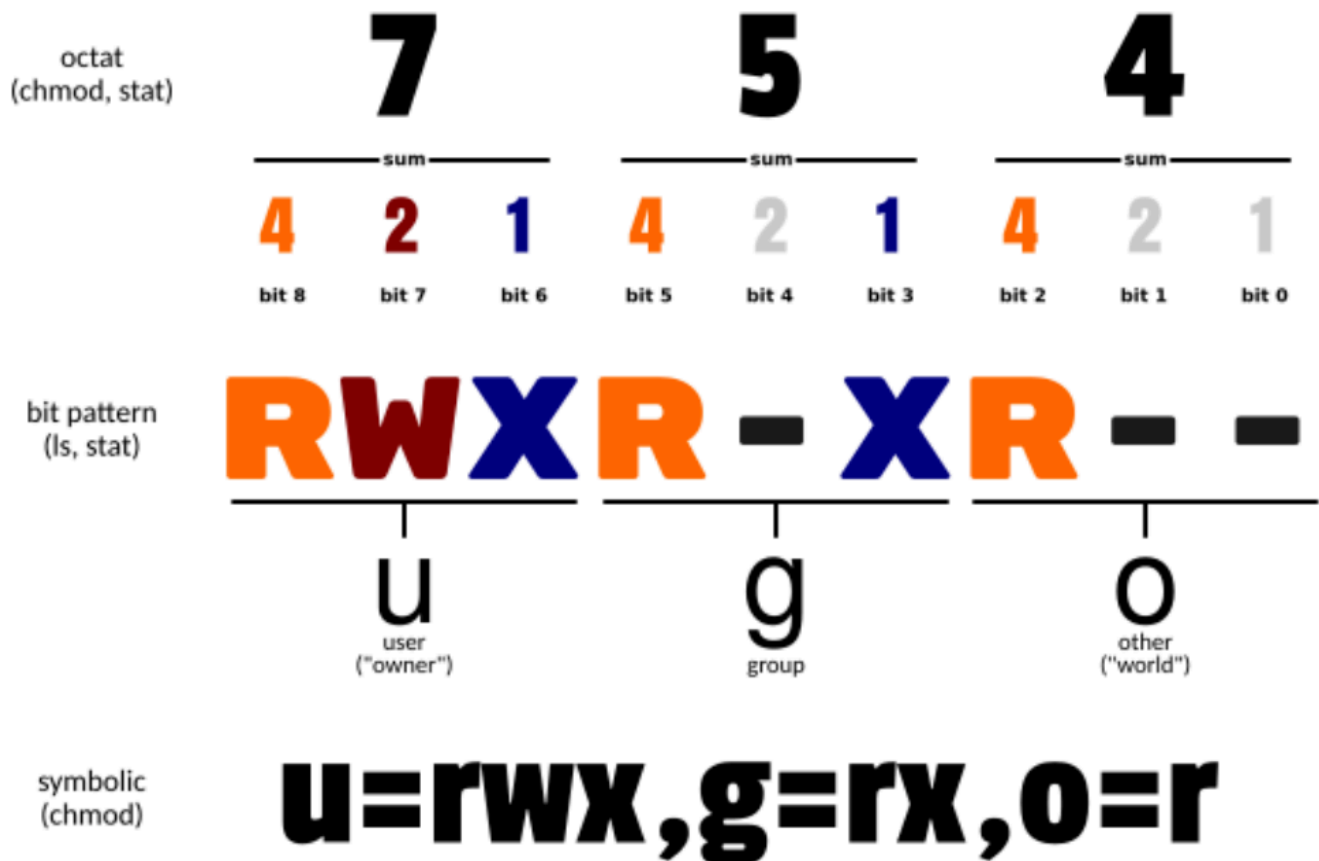
`chmod -R 755 public/` sets the permissions of the folder and subfolders to `rw-r-xr-x`

r: read, you can read the contents of this file, that is, you can open the file;

w: write, you can edit the content of this file, such as adding, deleting, and changing the file content;

x: execute, this file can be executed.

[OBJ]



Copy the root data file on cds to the server

```
xrdcp -d 1 -f root://xrootd-cms.infn.it//
```

The .h header file can be obtained using the mymultilep.root file generated by cmsRun in the crab step.

```
root /path/***.root
```

```
.ls //mkcands
```

```
mkcands->cd()
```

```
.ls //X_data
```

```
X_data->MakeClass("myntuple")
```

```

root [0]
Attaching file mymultilep.root as _file0...
(TFile *) 0x3e47710
[root [1] .ls
TFile**          mymultilep.root
TFile*           mymultilep.root
KEY: TDirectoryFile  mkcands;1      mkcands
[root [2] mkcands->cd()
(bool) true
[root [3] .ls
TDirectoryFile*   mkcands mkcands
KEY: TTree        X_data;8      X(3872) Data [current cycle]
KEY: TTree        X_data;7      X(3872) Data [backup cycle]
KEY: TTree        X_data;6      X(3872) Data [backup cycle]
[root [4] X_data->MakeClass("myntuple")

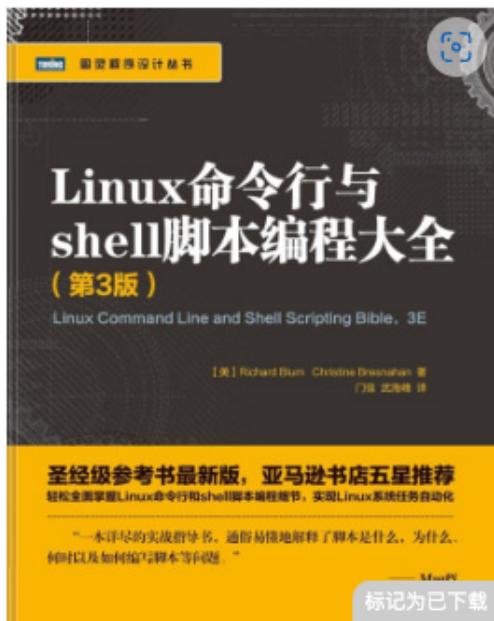
```

Lpc sometimes cannot use some commands
 source /cvmfs/cms.cern.ch/cmsset_default.sh
 source /cvmfs/cms.cern.ch/crab3/crab.sh

Modify path; clear;
 scramv1 b ProjectRename
 scramv1 b clean
 scramv1 b

Remove duplicate rows
 cat hh.txt | sort -g -k7 | uniq

disown -h %1 Invincible code, I would like to call it God! The job with serial number 1 is suspended in the background, and no stop code is sent when the connection is disconnected. Previously, if the background was suspended, the job would end if the connection was disconnected.



This article refers to this book. If you have any questions, please download and read it yourself.

[The Linux Command Line and Shell Scripting Bible](#)

END

written by zhuf

linux advanced and server management

Zero, some small knowledge to supplement the previous ones

- language

`locale` displays the current language

```
LANG="zh_CN.UTF-8"
```

```
LC_COLLATE="zh_CN.UTF-8"
```

```
LC_CTYPE="zh_CN.UTF-8"
```

```
LC_MESSAGES="zh_CN.UTF-8"
```

```
LC_MONETARY="zh_CN.UTF-8"
```

```
LC_NUMERIC="zh_CN.UTF-8"
```

```
LC_TIME="zh_CN.UTF-8"
```

```
LC_ALL=
```

`LANG=en_US.utf8` Modify the output information to English

`export LC_ALL=en_US.utf8` Synchronously update LC_ALL

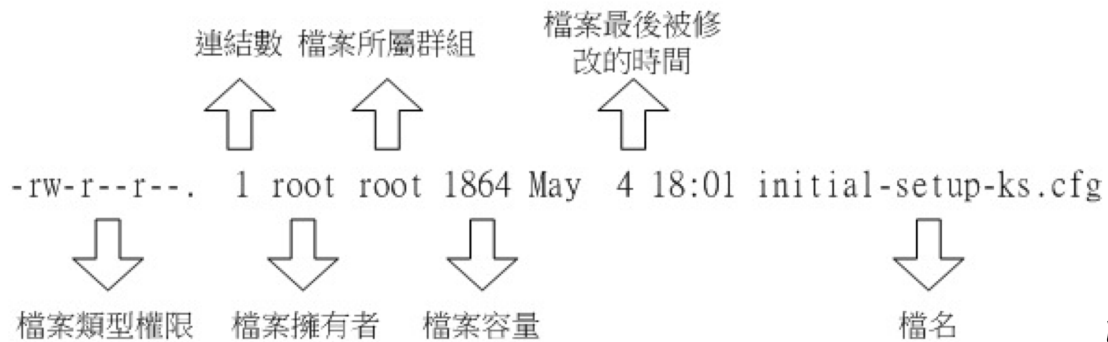
- man command

When you don't know how to use a command, just click "man" and you will know that the displayed manual is usually divided into the following parts:

Codename	Content
NAME	A short command description. In addition to the command, there are some numbers in brackets in the upper left corner. These are more important: 1. User-executable commands or files; 5. Configuration files or the format of a certain file; 8. Commands available to system administrators
SYNOPSIS	Short command syntax
DESCRIPTION	Full description
COMMANDS	When the program is executed, this command can be issued in the program

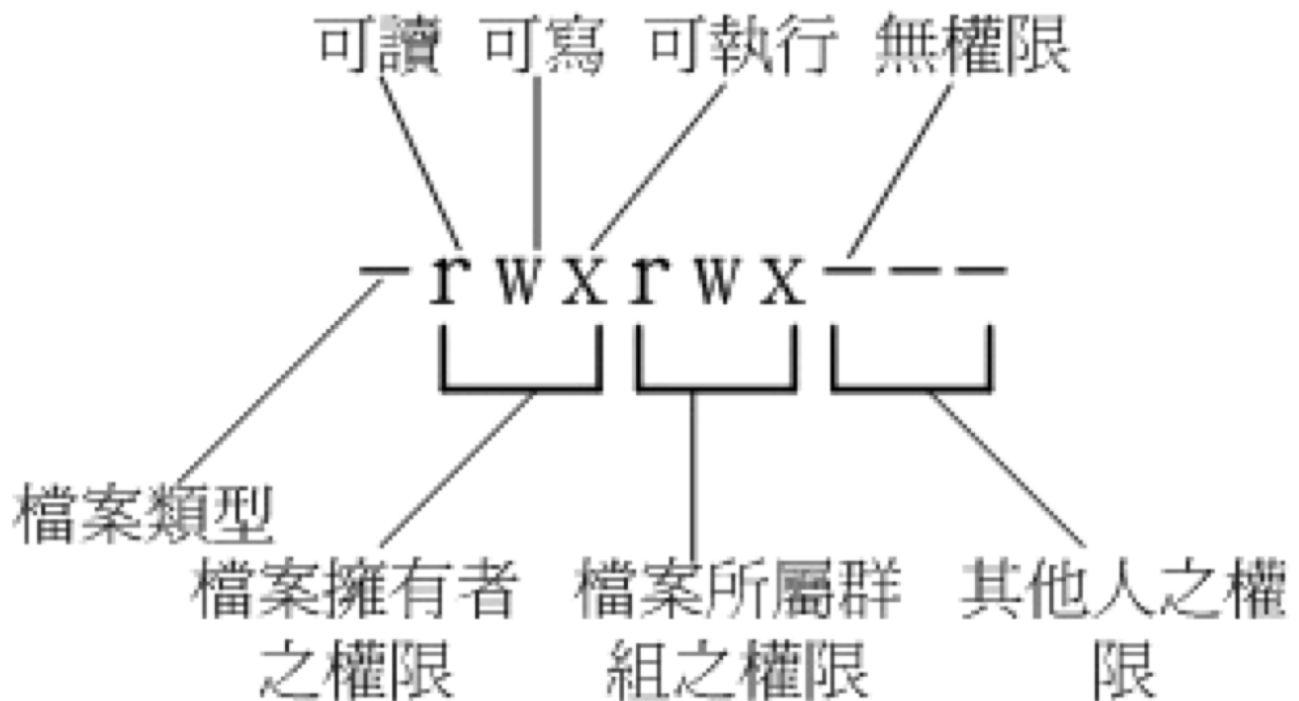
- File permissions

Each file in Linux has three identity permissions: User, Group, and Others. Check the file permissions with `ls -al`



图

5.2.1、文件属性的示意图



E

File type [d] directory [-] file [l] link file

- Change file attributes

- chgrp changes the group to which the file belongs

`/etc/group` is the name of the group that exists. Use `chgrp [group name] [file name]` to copy the corresponding file

- chown changes file owner

Users exist in `/etc/passwd`. Use `chown -R [user name] [dirname/filename]` OR `chown -R [user name]:[group name] [dirname/filename]` to modify the group and user affiliation. -R represents recursion.

- `chmod` changes file permissions
File permissions are represented by the sum of numbers r:4 w:2 x:1
Some commonly used permission numbers
755 Make the file executable but prevent others from modifying it
740 Keep files from being seen by others
- some instructions
`who` Can see who is working secretly
`df` Usage of equipment in the system

1. Linux files and directories

1.1/(root)root directory

Directory	Place file contents
<code>/bin</code>	bin places some operating instructions, such as <code>cat</code> , <code>mv</code> , <code>cp</code>
<code>/boot</code>	Place boot configuration file
<code>/etc</code>	System configuration file, including personnel account passwords
<code>/lib</code>	Place function library
<code>/tmp</code>	As its name suggests, it is a temporary file
<code>/usr</code>	(Unix Software Resource) Some system default software, similar to the C drive of window
<code>/dev</code>	Every device in Linux is treated as a file. This is the device file.

There are many software sources; you can use some [regional mirrors](#).

`/etc/apt/sources.list` OR `/etc/apt/sources.list.d/` contains the software source link

`sudo apt-get update` Update software sources through this command

1.2 \$PATH executable file path variable

When executing a command "ls", the system will search for an executable file named ls in each defined directory according to the PATH setting to execute the command.

Take \$PATH `/usr/local/bin:/usr/sbin:/usr/bin:/root/bin` as an example. If you use root permissions to move `/bin/ls` to the `/root` folder, you cannot execute the `ls` command in this case. Only when you use the command

`PATH="${PATH}:/root"` to add the ls path under `/root` to \$PATH can you execute the ls command correctly.

If there are two ls, the ls searched first in the directory in \$PATH will be executed.

Not finished...